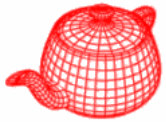
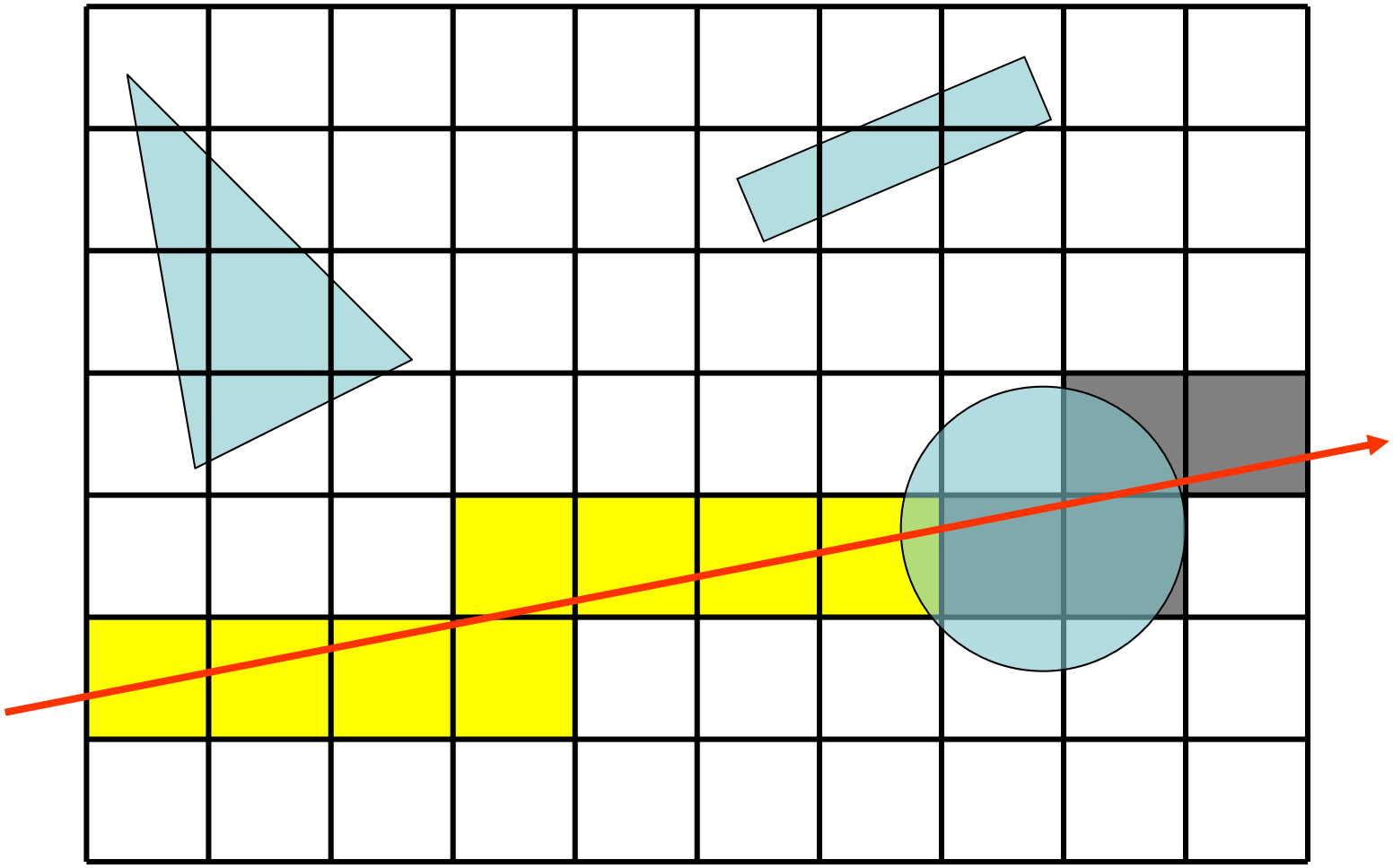
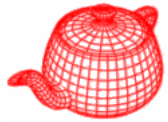
Grid accelerator



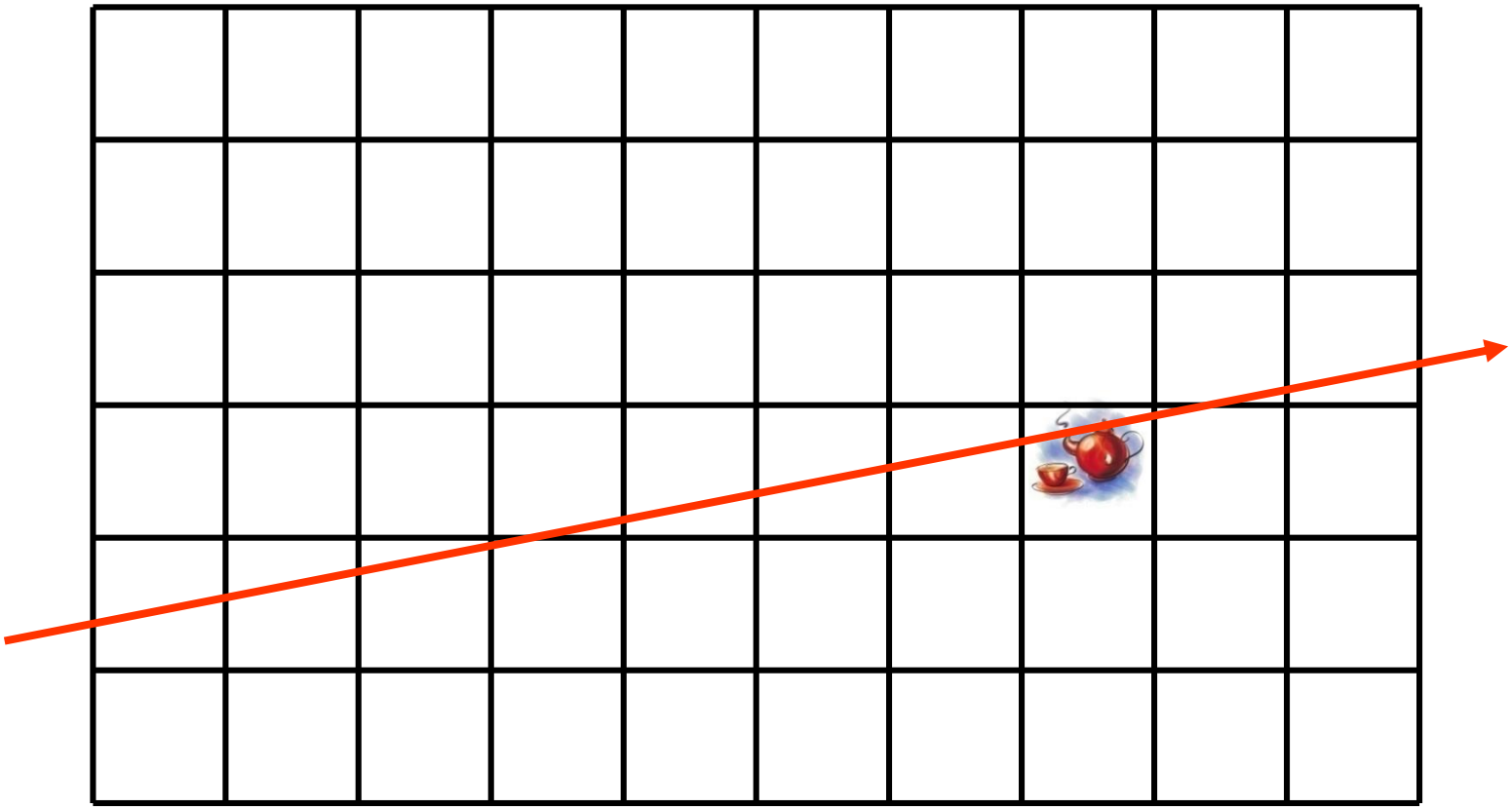
- Uniform grid



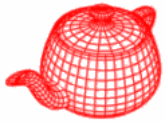
Teapot in a stadium problem



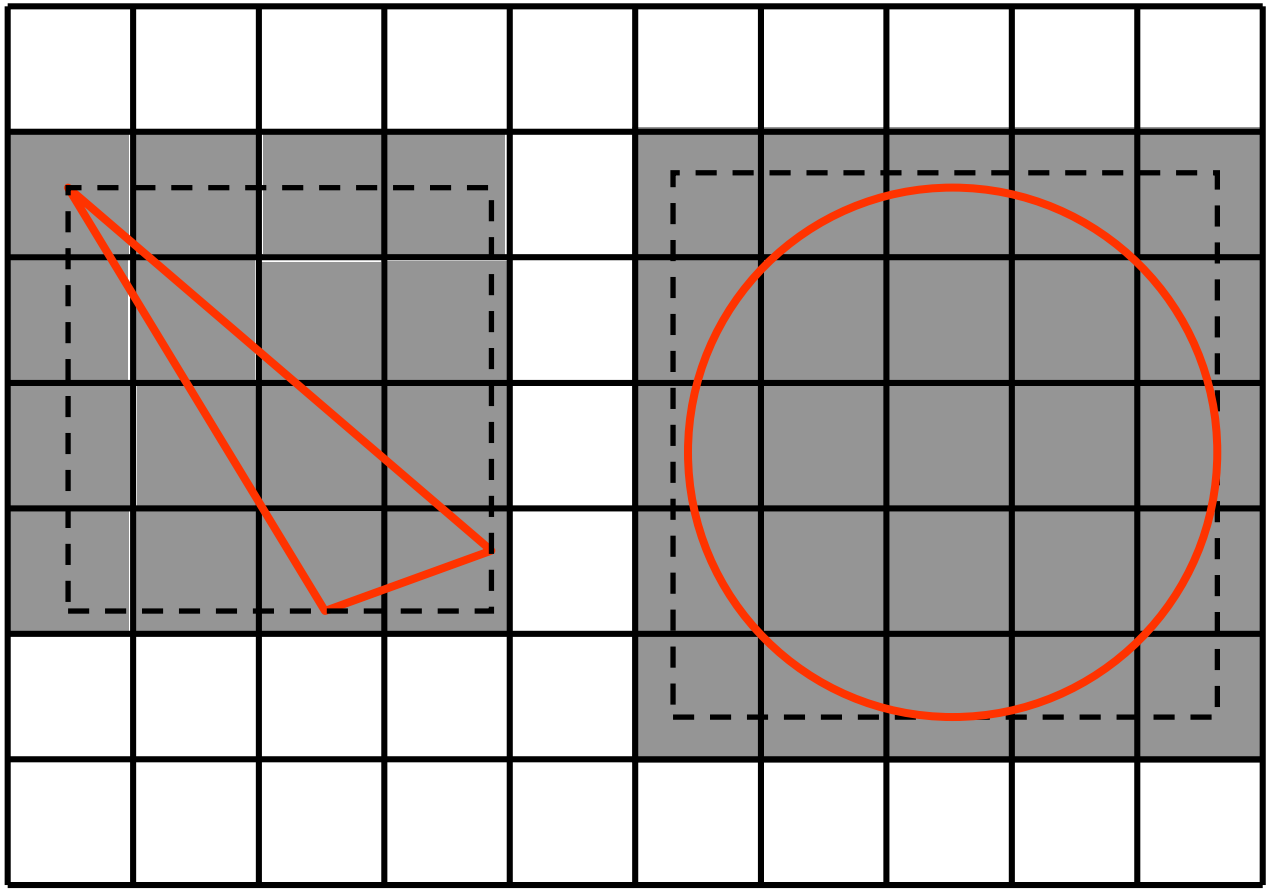
- Not adaptive to distribution of primitives.
- Have to determine the number of voxels. (problem with too many or too few)



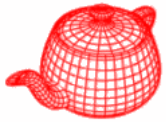
Grid construction



```
for (u_int i=0; i<prims.size(); ++i) {
  <Find voxel extent of primitive>
  <Add primitive to overlapping voxels>
}
```

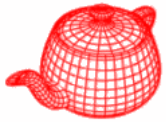


Grid intersection

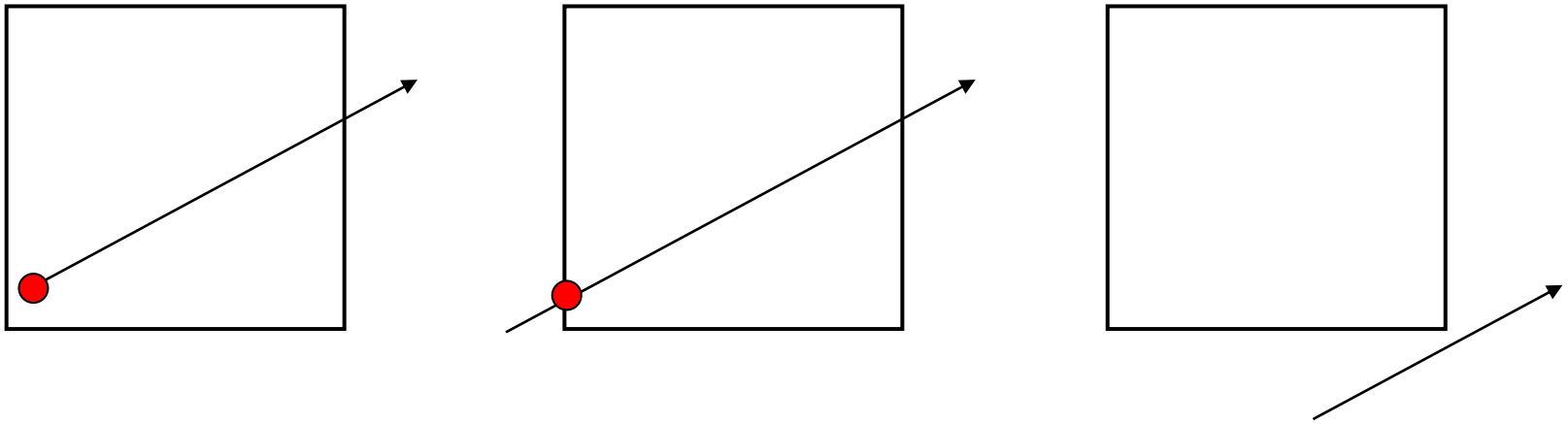


```
bool GridAccel::Intersect(
        Ray &ray, Intersection *isect) {
  <Check ray against overall grid bounds>
  <Set up 3D DDA for ray>
  <Walk ray through voxel grid>
}
```

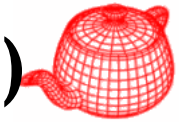
Check against overall bound



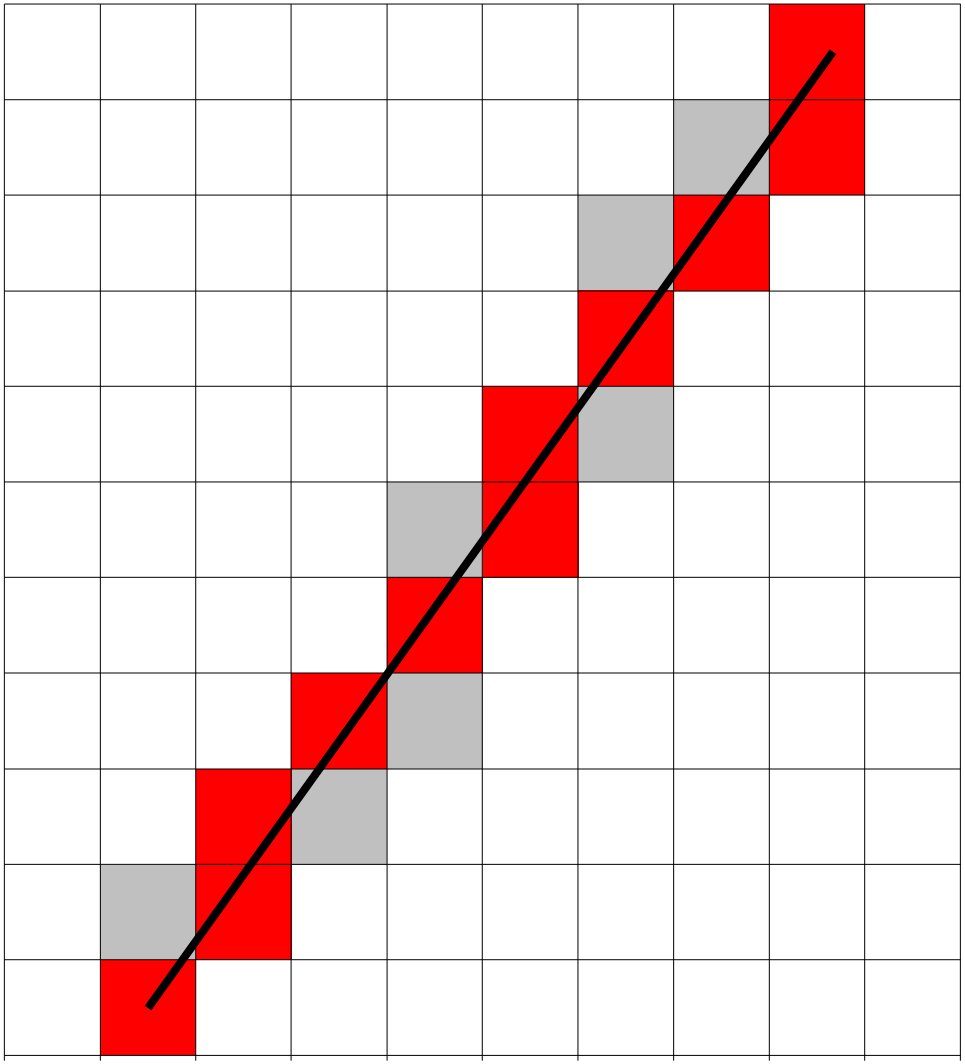
```
float rayT;
if (bounds.Inside(ray(ray.mint)))
   rayT = ray.mint;
else if (!bounds.IntersectP(ray, &rayT))
   return false;
Point gridIntersect = ray(rayT);
```



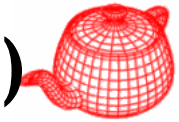
Set up 3D DDA (Digital Differential Analyzer)

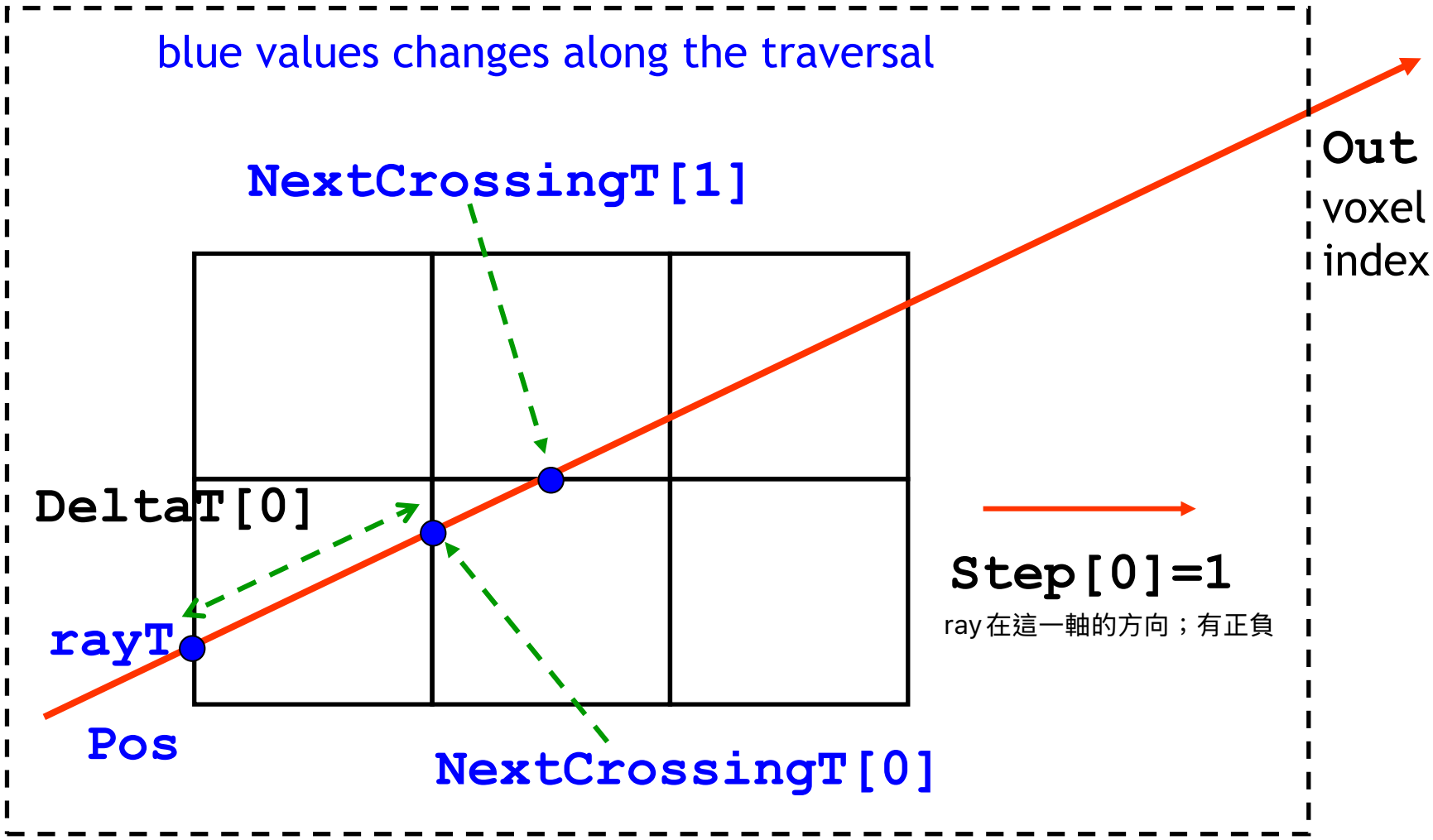


- Similar to Bresenhm's line drawing algorithm



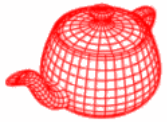
Set up 3D DDA (Digital Differential Analyzer)



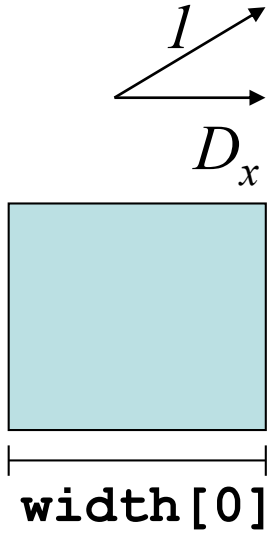


DeltaT: the distance change when voxel changes 1 in that direction

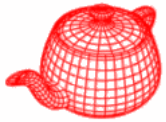
Set up 3D DDA



```
for (int axis=0; axis<3; ++axis) {
  Pos[axis]=posToVoxel(gridIntersect, axis);
  if (ray.d[axis]>=0) {
    NextCrossingT[axis] = rayT+
    (voxelToPos(Pos[axis]+1,axis)-gridIntersect[axis])
    /ray.d[axis];

    DeltaT[axis] = width[axis] / ray.d[axis];
    Step[axis] = 1;
    Out[axis] = nVoxels[axis];
  } else {
    ...
    Step[axis] = -1;
    Out[axis] = -1;
  }
}
```

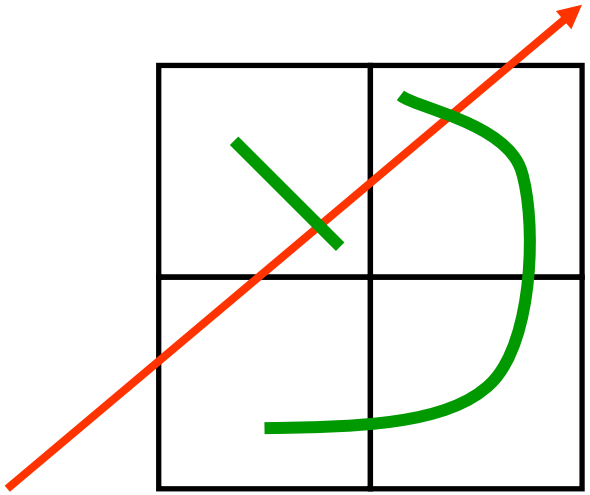


Walk through grid

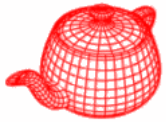


```
for (;;) {
  *voxel=voxels[offset(Pos[0],Pos[1],Pos[2])];
  if (voxel != NULL)
    hitSomething |=
            voxel->Intersect(ray,isect,rayId);
  <Advance to next voxel>
}
return hitSomething;
```

Do not return; cut tmax instead
Return when entering a voxel
that is beyond the closest
found intersection.

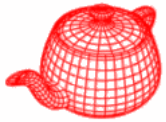


Advance to next voxel



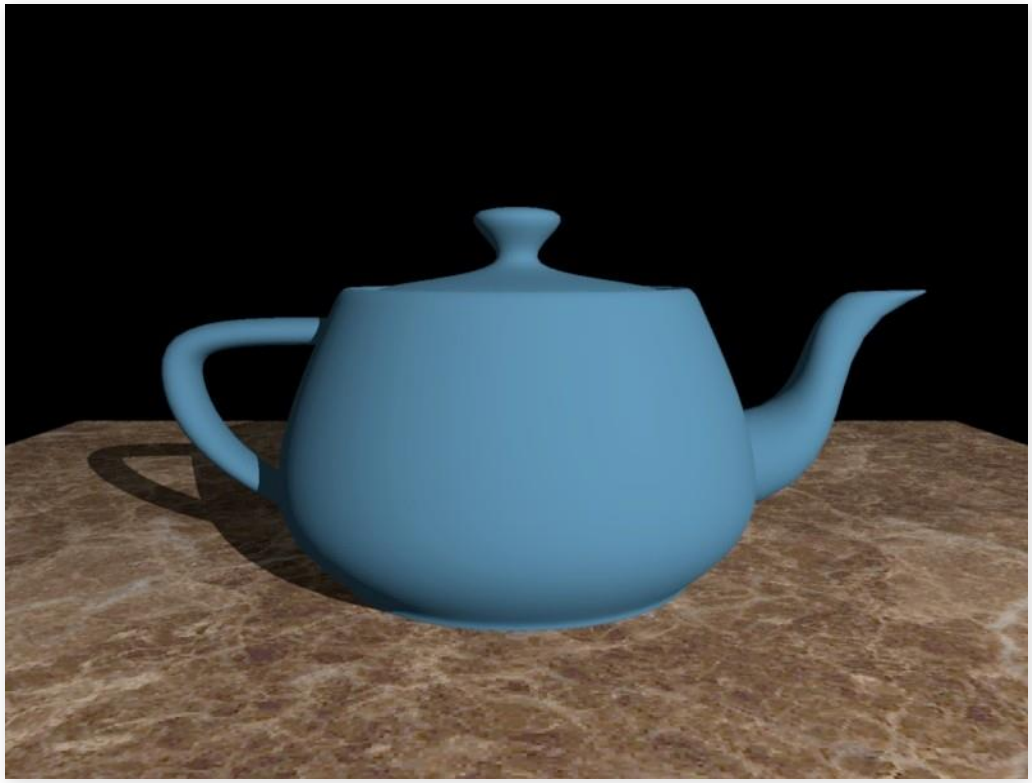
```
int bits=((NextCrossingT[0]<NextCrossingT[1])<<2) +
         ((NextCrossingT[0]<NextCrossingT[2])<<1) +
         ((NextCrossingT[1]<NextCrossingT[2]));
const int cmpToAxis[8] = { 2, 1, 2, 1, 2, 2, 0, 0 };

int stepAxis=cmpToAxis[bits];

if (ray.maxt < NextCrossingT[stepAxis]) break;

Pos[stepAxis]+=Step[stepAxis];

if (Pos[stepAxis] == Out[stepAxis]) break;

NextCrossingT[stepAxis] += DeltaT[stepAxis];
```

conditions



x<y	x<z	y<z		
0	0	0	x≥y≥z	2
0	0	1	x≥z>y	1
0	1	0	-	
0	1	1	z>x≥y	1
1	0	0	y>x≥z	2
1	0	1	-	
1	1	0	y≥z>x	0
1	1	1	z>y>x	0

Statistics of a simple example



6321 triangles
800 x 600 image

Brute force:
6321 inters. tests /ray

Regular grid:
44.86 inters. tests /ray

2-level grid:
12.05 inters. tests /ray

BVH:
2.6 inters. tests/ray

Slides credit: Wojciech Jarosz